MOBILE APP DEVELOPMENT 700

RICHFIELD
richfield.ac.za

Name and Surname: Tracey Greenwood-Selby

Student ITS No: 402411848

Qualification: BSc IT **Year of Study:** 3 **Semester:** 2

Assignment due date: 23/10/26 **Date submitted:** 23/10/26

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature:  **Date:** 23/10/26

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

Table of Contents

1

1. Introduction 3

2. System Design 3

2.1 Screen flow 3

2.2 Database model 4

2.3 Data handling and strict matching 4

3. Screenshots of Application Outputs 5

4. Key Code Snippets 14

4.1 Validating quantity 14

4.2 SQLite persistence 14

4.3 Recipe eligibility 14

4.5 Expiry-date compatibility 14

5. Challenges and Solutions 15

5.1 Missing layout identifier 15

5.2 Android API compatibility 15

5.3 Accessibility and maintainability 15

6. Conclusion and Reflection 16

7. Reference List 16

1. Introduction

Food waste can occur when people overlook ingredients already available at home or cannot identify meals that use those ingredients. Smart Pantry Manager is an Android application intended for home cooks and students who want to keep a record of pantry items and prepare meals without purchasing missing ingredients.

The application was developed in Java using Android Studio. It stores pantry and recipe data in an on-device SQLite database. Its central rule is strict matching: a recipe is suggested only when every required ingredient is present in a sufficient quantity. The interface provides pantry management, suggested recipes, recipe details and settings.

2. System Design

2.1 Screen flow

The Pantry List acts as the primary entry point. Users can add or edit ingredients, view suggestions, or open Settings. A recipe selection opens its details. Back buttons support returning to the preceding screen.

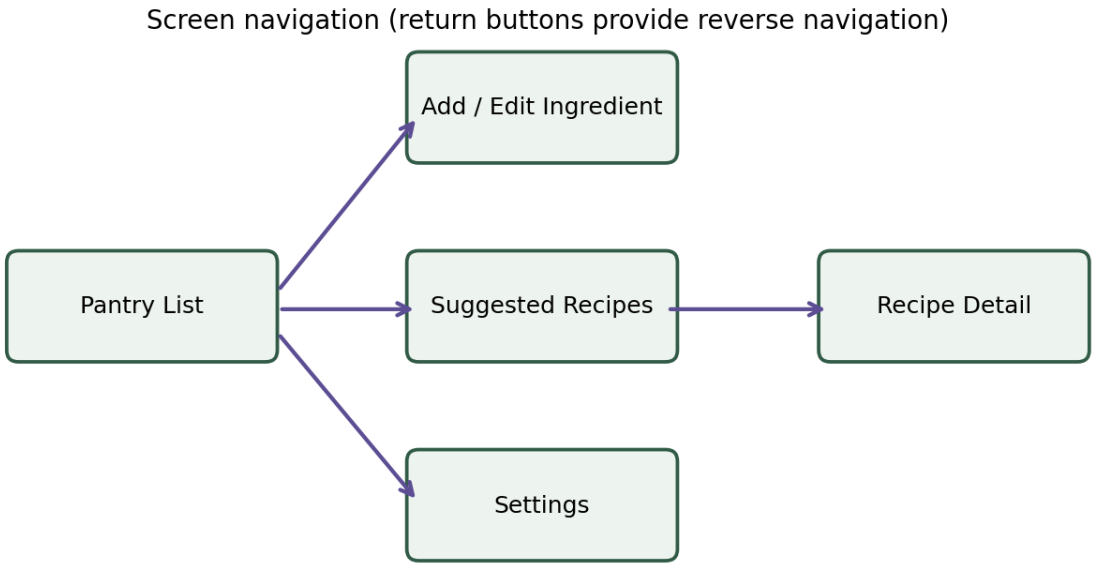


Figure 1. Screen-flow diagram for the five application screens.

2.2 Database model

SQLiteOpenHelper manages the local SmartPantry.db database. The ingredients table stores pantry records. The recipes table stores seeded recipes, and recipe_ingredients stores their requirements. Each recipe may have multiple recipe_ingredients records. Pantry records are compared with these requirements rather than linked by a foreign key.

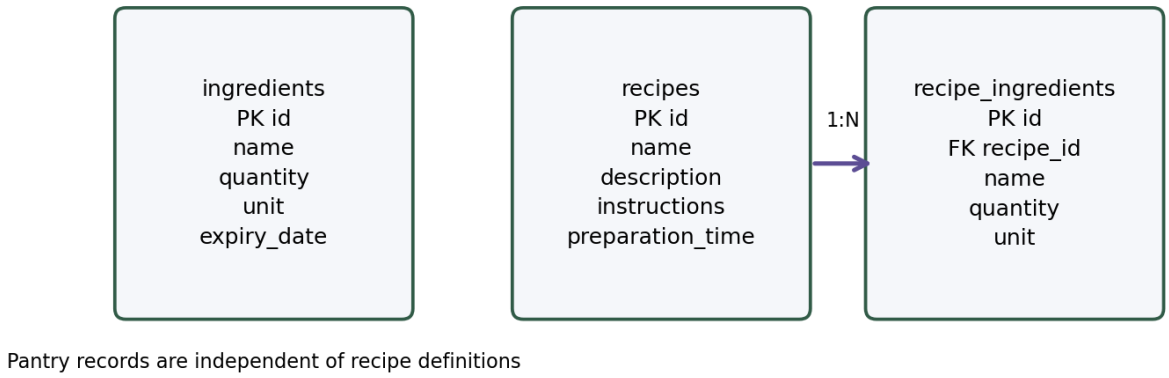


Figure 2. Simplified entity-relationship model based on the implemented database tables.

2.3 Data handling and strict matching

The pantry supports create, read, update and delete operations. On first creation or upgrade, the recipe collection is seeded into SQLite. RecipeMatcher compares normalised ingredient names, checks available quantities and compatible units, and excludes expired ingredients. Recipes are shown only when all requirements pass. Tests covered missing ingredients, insufficient quantities, unit conversion, case differences and expiry dates.

3. Screenshots of Application Outputs

All images below were captured from the running Android emulator during application testing. Screenshots reflect different test states; the ingredient count and quantities can therefore vary.

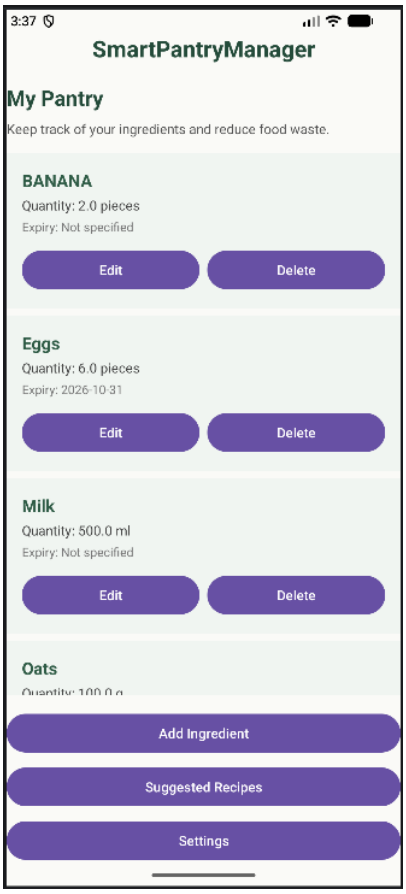


Figure 3. Pantry List: ingredients and available Edit, Delete and navigation actions.

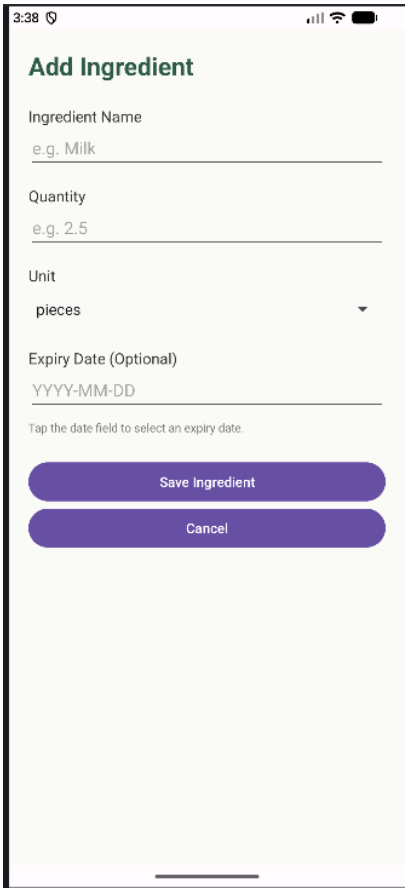


Figure 4. Add Ingredient: fields for name, quantity, unit and optional expiry date.

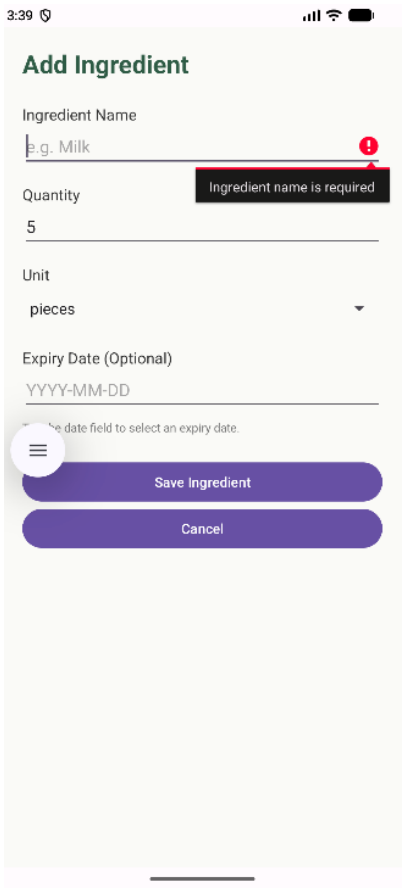


Figure 5. Input validation: an empty ingredient name produces a visible error.

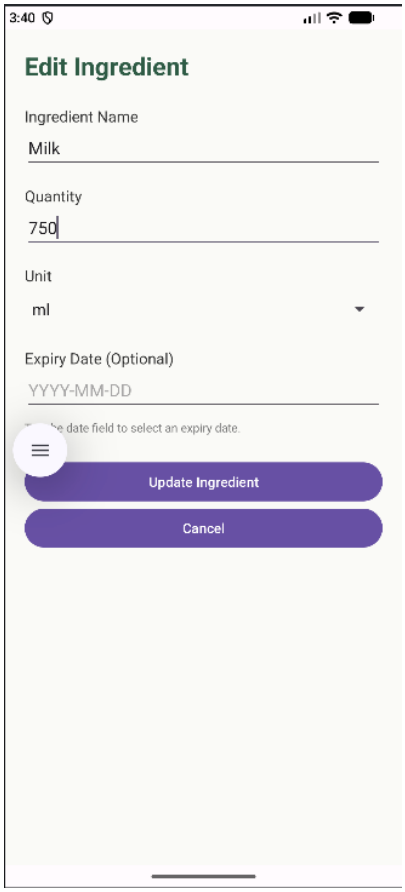


Figure 6. Edit Ingredient: existing Milk record with a revised quantity of 750 ml before saving.

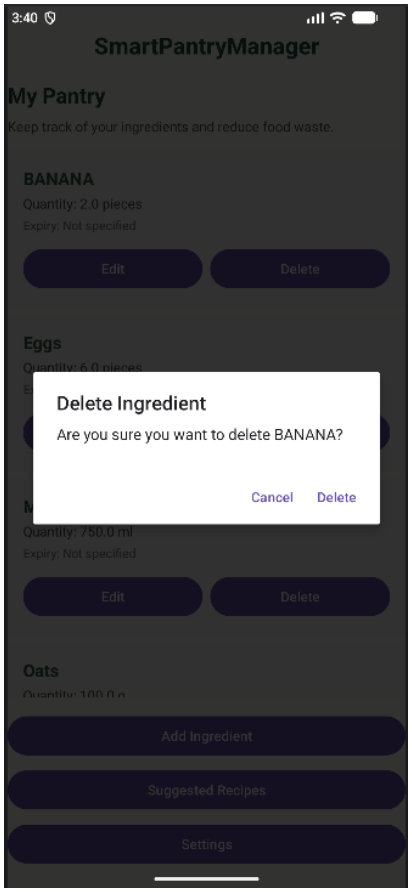


Figure 7. Delete confirmation: user can cancel or confirm removing BANANA.

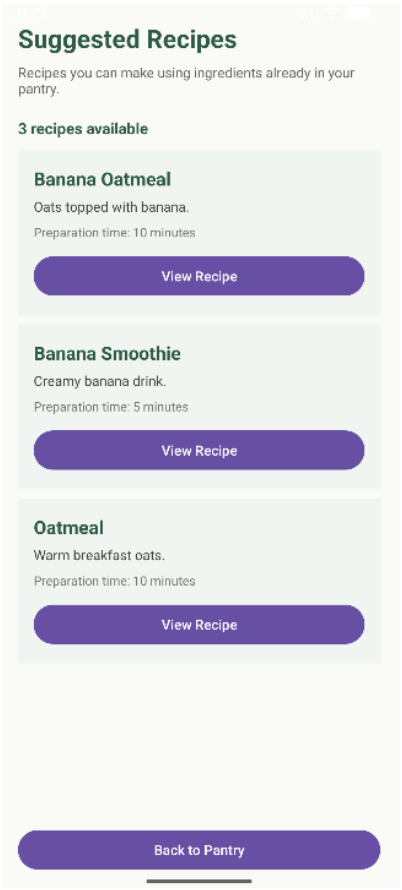


Figure 8. Suggested Recipes: three currently matching recipes are displayed.

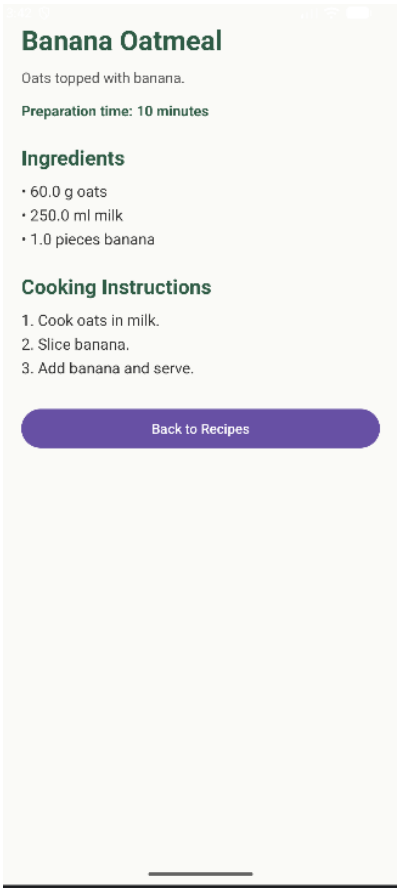


Figure 9. Recipe Detail: Banana Oatmeal ingredients, quantities and preparation steps.

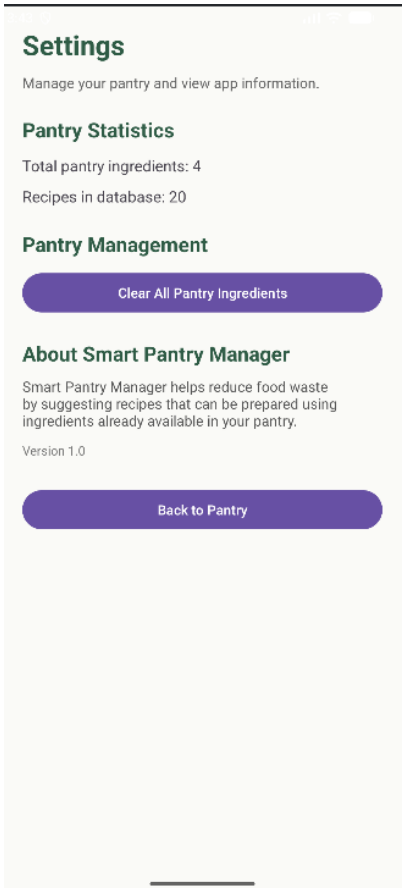


Figure 10. Settings: pantry statistics, recipe count and clear-pantry option.

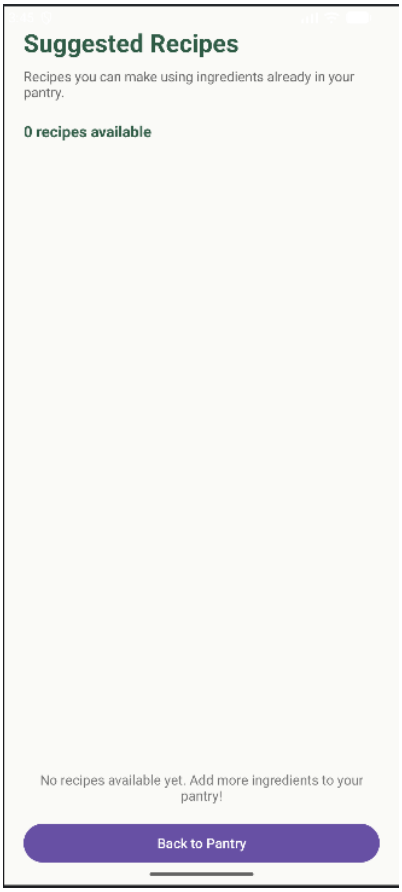


Figure 11. Empty recipe state: zero matching recipes and helpful user feedback.

4. Key Code Snippets

The following are explanatory extracts reconstructed from the reviewed implementation and development discussion. Before submission, compare each extract with the current GitHub source and replace it with the exact corresponding lines if different.

4.1 Validating quantity

```
quantity = Double.parseDouble(quantityText);
if (!Double.isFinite(quantity) || quantity <= 0) {
    editQuantity.setError("Quantity must be greater than zero");
    return;
}
```

This check rejects zero, negative and non-finite values before saving. The surrounding try/catch also handles invalid numeric text.

4.2 SQLite persistence

```
databaseHelper.addIngredient(ingredient);
databaseHelper.updateIngredient(ingredient);
databaseHelper.deleteIngredient(ingredientId);
```

SQLiteOpenHelper provides the persistent storage layer for CRUD operations. The exact method signatures should be verified against DatabaseHelper.java.

4.3 Recipe eligibility

```
RecipeMatcher.findMatchingRecipes(recipes, pantryIngredients);
```

The matcher returns recipes that satisfy every required ingredient after name normalisation, quantity checks, supported unit conversion and expiry filtering. This line illustrates the matcher call; verify the precise call site.

4.5 Expiry-date compatibility

```
SimpleDateFormat dateFormat =
    new SimpleDateFormat("yyyy-MM-dd", Locale.US);
dateFormat.setLenient(false);
Date expiry = dateFormat.parse(expiryDate.trim());
```

This parsing approach supports the application's Android minimum API level without relying on LocalDate.parse(), which was flagged by Android Lint.

5. Challenges and Solutions

5.1 Missing layout identifier

A Gradle build failed because MainActivity referred to R.id.main while the replacement root layout no longer declared that ID. The android:id="@+id/main" attribute was restored to the root LinearLayout, after which the build succeeded.

5.2 Android API compatibility

Android Lint identified a LocalDate.parse() call that requires API level 26 or desugaring, while the project minimum is API 24. The expiry-date logic was updated to use SimpleDateFormat and Date. A subsequent debug build and Lint run completed successfully.

5.3 Accessibility and maintainability

Lint warned that the clickable expiry-date field lacked keyboard focus support and that several interface strings were hardcoded. The date field was made focusable and configured for date input, and the Pantry Home screen labels were moved into strings.xml. The date picker and navigation were retested afterward.

6. Conclusion and Reflection

Smart Pantry Manager demonstrates Android Activities, Intents, XML layouts, RecyclerView adapters, input validation and persistent SQLite data. The central technical challenge was implementing strict recipe eligibility rather than showing partial matches. The completed emulator tests covered six matching scenarios, seven validation cases and three persistence cases, alongside screen-navigation checks.

The project strengthened practical understanding of Java Android development, database CRUD, UI debugging and static analysis. Future improvements could include expiry reminders, additional recipe categories, automated tests and more extensive string-resource localisation. Before final submission, the report's example code extracts should be reconciled with the final repository and the video demonstration recorded.

7. Reference List

Git (n.d.) *Git documentation*. Available at: https://git-scm.com/doc (Accessed: 22 October 2026).

Google (n.d.) *Create dynamic lists with RecyclerView*. Android Developers. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 22 October 2026).

Google (n.d.) *Save data using SQLite*. Android Developers. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 22 October 2026).

Material Design (n.d.) *Text fields*. Available at: https://m1.material.io/components/text-fields.html (Accessed: 22 October 2026).

Oracle (n.d.) *Class Double (Java Platform SE 8)*. Java Documentation. Available at: https://docs.oracle.com/javase/8/docs/api/java/lang/Double.html (Accessed: 22 October 2026).

SQLite (2026) *SQLite foreign key support*. Available at: https://www.sqlite.org/foreignkeys.html (Accessed: 22 October 2026).